Pipelining Hazards

Micro-Architecture

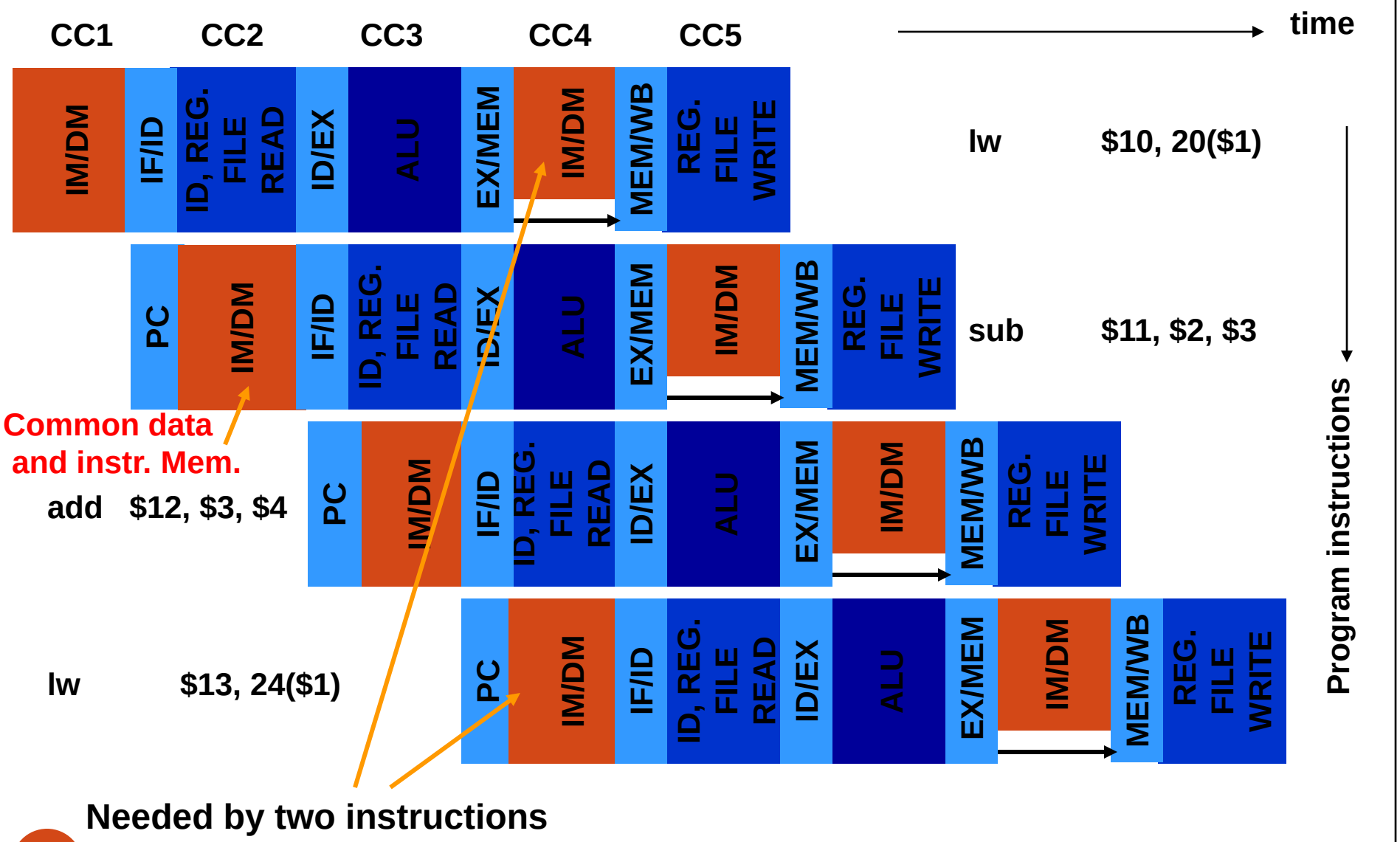
Pipeline Hazards

- Definition: *Hazard in a pipeline is a situation in which the next instruction cannot complete execution one clock cycle after completion of the present instruction.*
- Three types of hazards:
  - Structural hazard (resource conflict)
  - Data hazard
  - Control hazard

Structural Hazard

- Two instructions cannot execute due to a resource conflict.
- Example: Consider a computer with a common data and instruction memory. The fourth cycle of a *lw* instruction requires memory access (memory read) and at the same time the first cycle of the fourth instruction requires instruction fetch (memory read). This will cause a memory resource conflict.

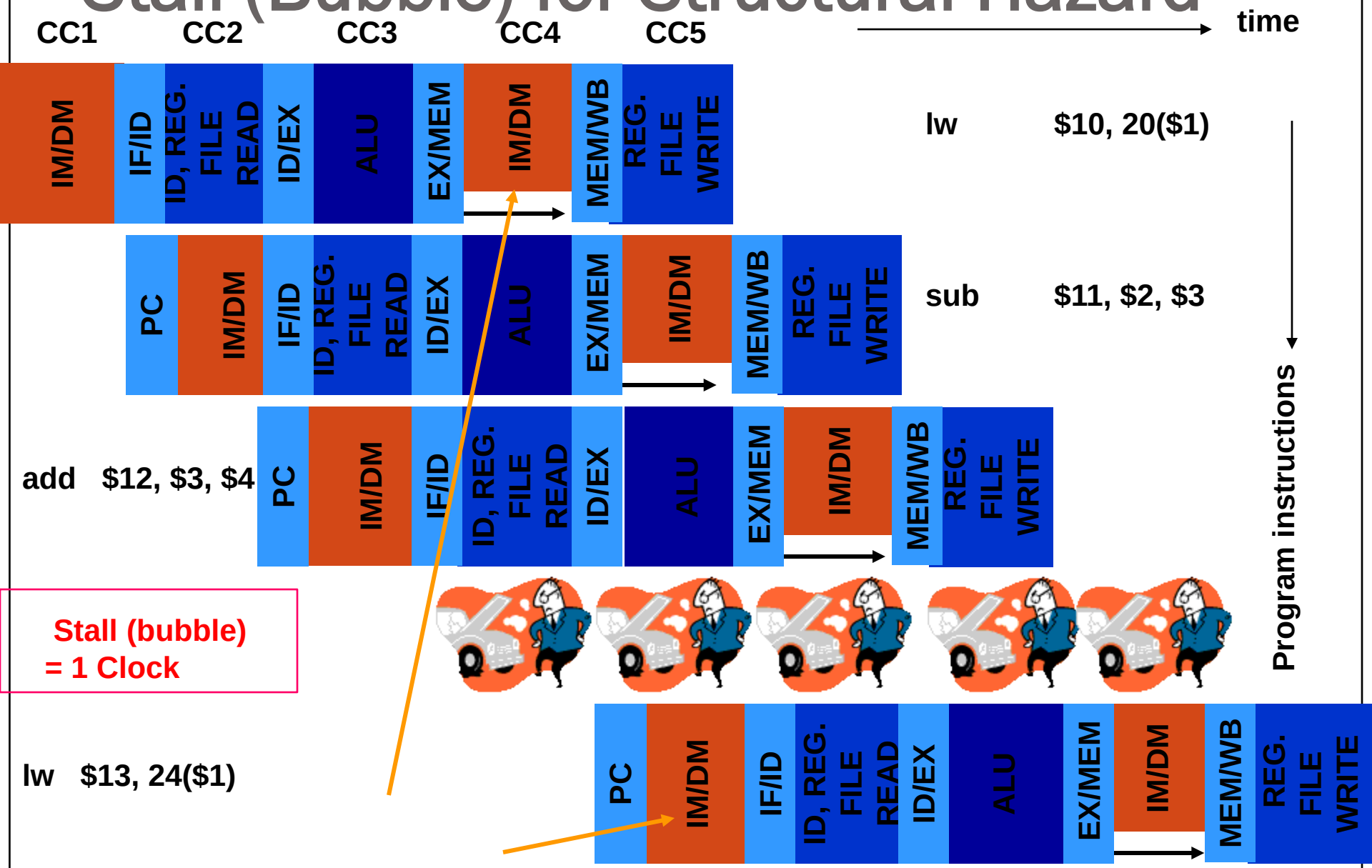
Example of Structural Hazard



Possible Remedies for Structural Hazards

- Provide duplicate hardware resources in datapath.
- Control unit or compiler can insert delays (no-op cycles) between instructions. This is known as pipeline *stall* or *bubble*.

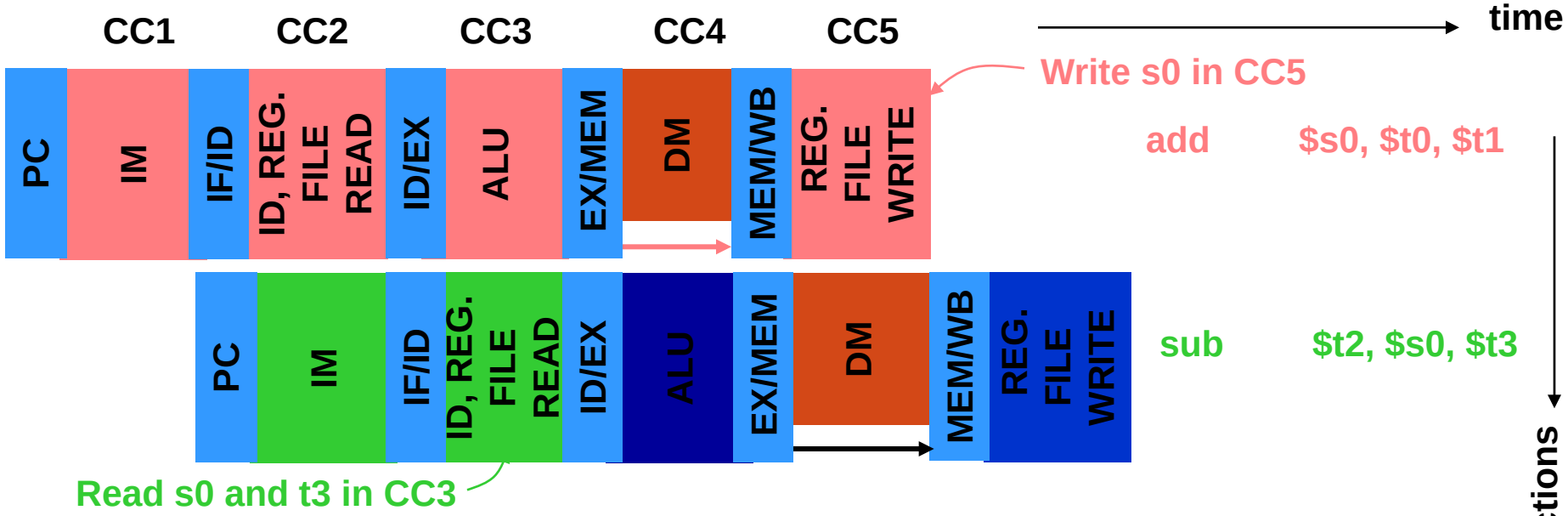
Stall (Bubble) for Structural Hazard



Data Hazard

- Data hazard means that an instruction cannot be completed because the needed data, being generated by another instruction in the pipeline, is not available.
- Example: consider two instructions:
  - add $s0, $t0, $t1
  - sub $t2, $s0, $t3 # needs $s0

Example of Data Hazard



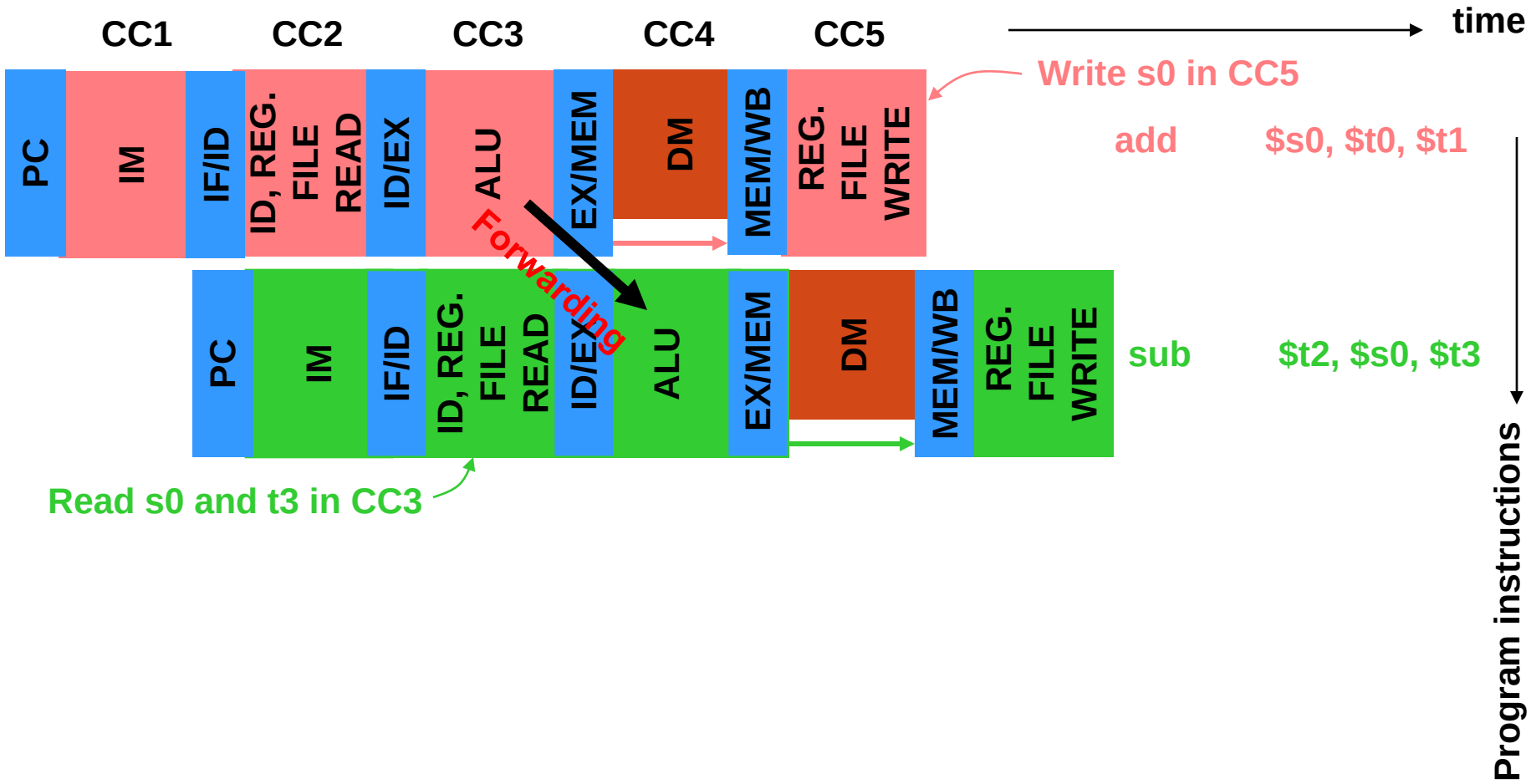
We need to read s0 from reg file in cycle 3
But s0 will not be written in reg file until cycle 5

However, s0 will only be used in cycle 4
And it is available at the end of cycle 3

Forwarding or Bypassing

- Output of a resource used by an instruction is forwarded to the input of some resource being used by another instruction.
- Forwarding can eliminate some, but not all, data hazards.

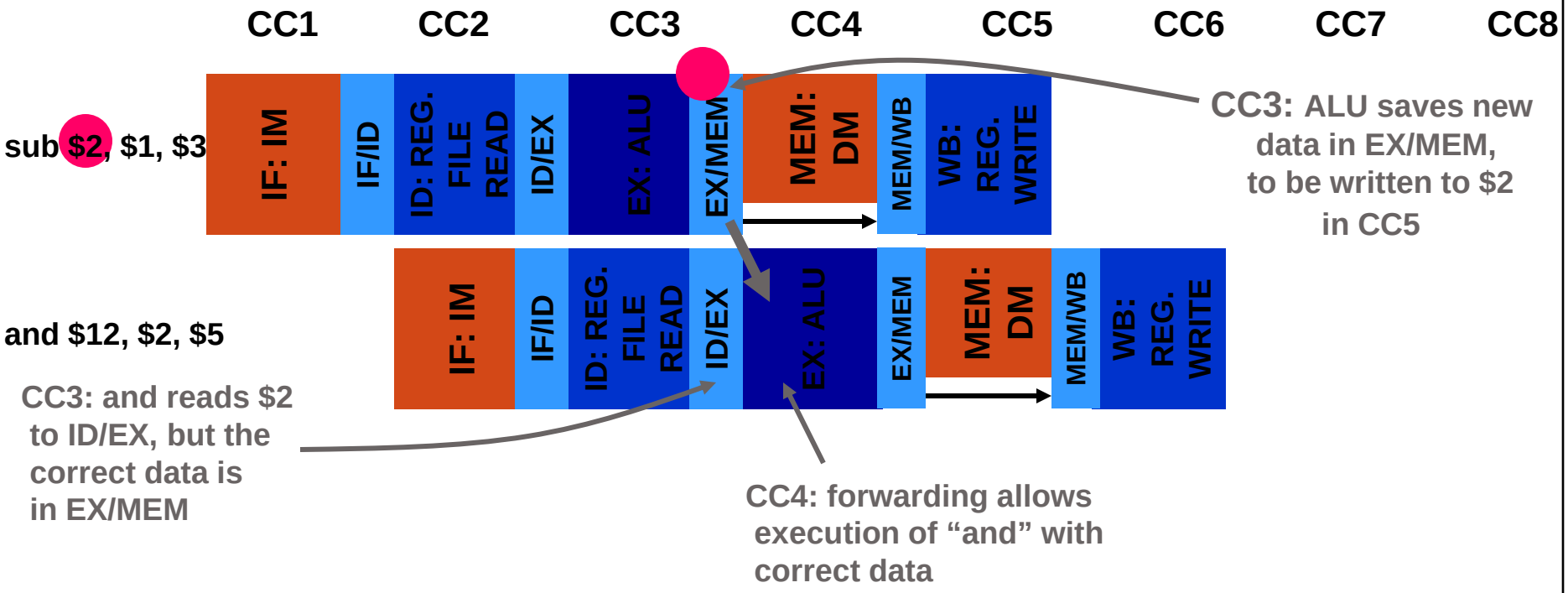
Forwarding for Data Hazard



Forwarding for Data Hazard

- Data hazard:

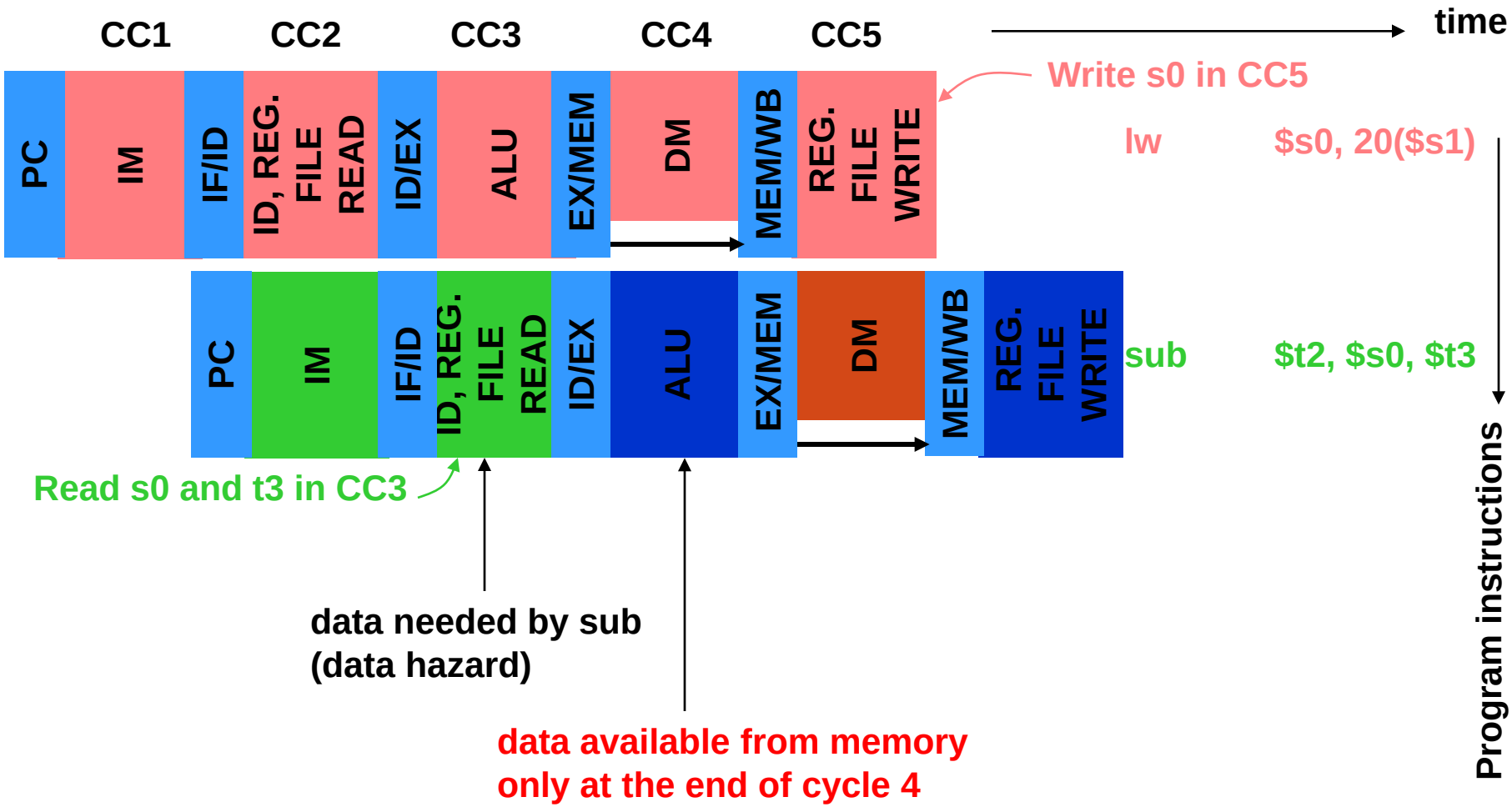
```
sub   $2, $1, $3      # computes result in CC3, writes in $2 in CC5 and
      $12, $2, $5     # reads $2 in CC3, adds in CC4
```



Forwarding Hardware

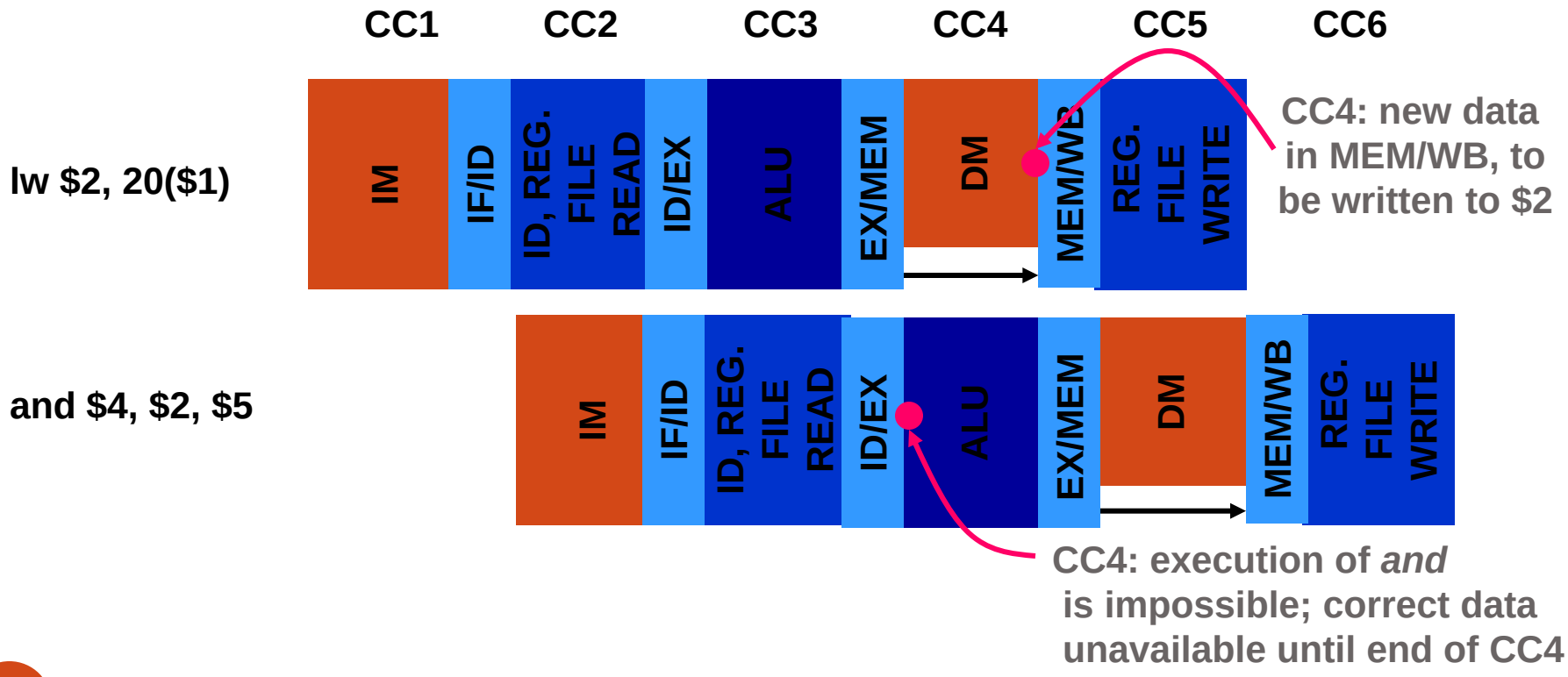
- A forwarding unit is added to execute (ALU) cycle hardware.
- Functions of forwarding unit:
  - Hazard detection
  - Forward correct data to ALU
- Inputs to forwarding unit:
  - Source registers of the instruction in EX
  - Destination registers of instructions in DM and WB
- Outputs of forwarding unit: multiplexer controls to route correct data to the ALU.

Forwarding Alone May Not Work

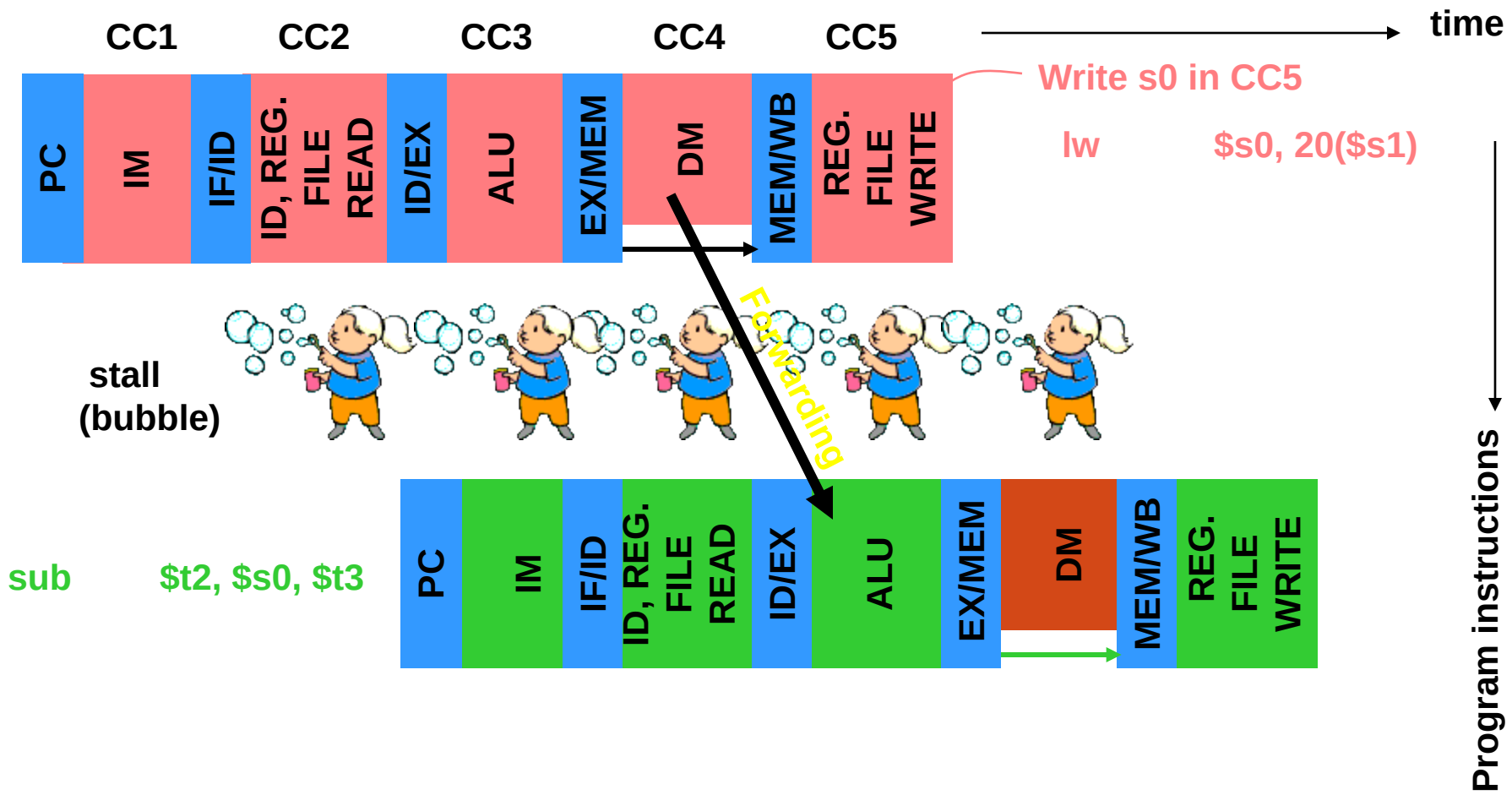


Use Stall and Forwarding

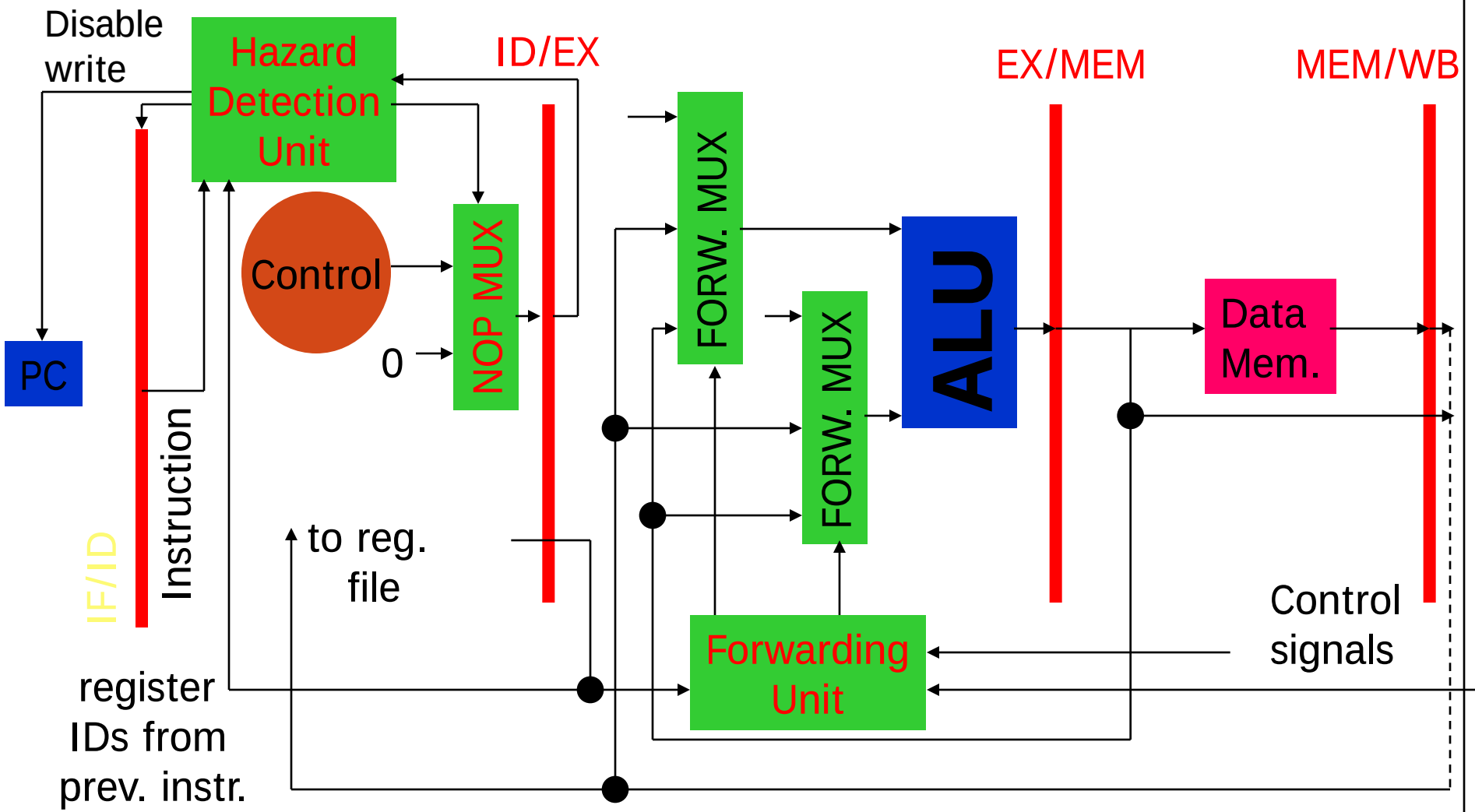
- Delay next instruction by sending nop through pipeline.
- Necessary when hazard not resolved by forwarding.



Use Stall and Forwarding



Hazard Detection Unit Hardware



Resolving Hazards

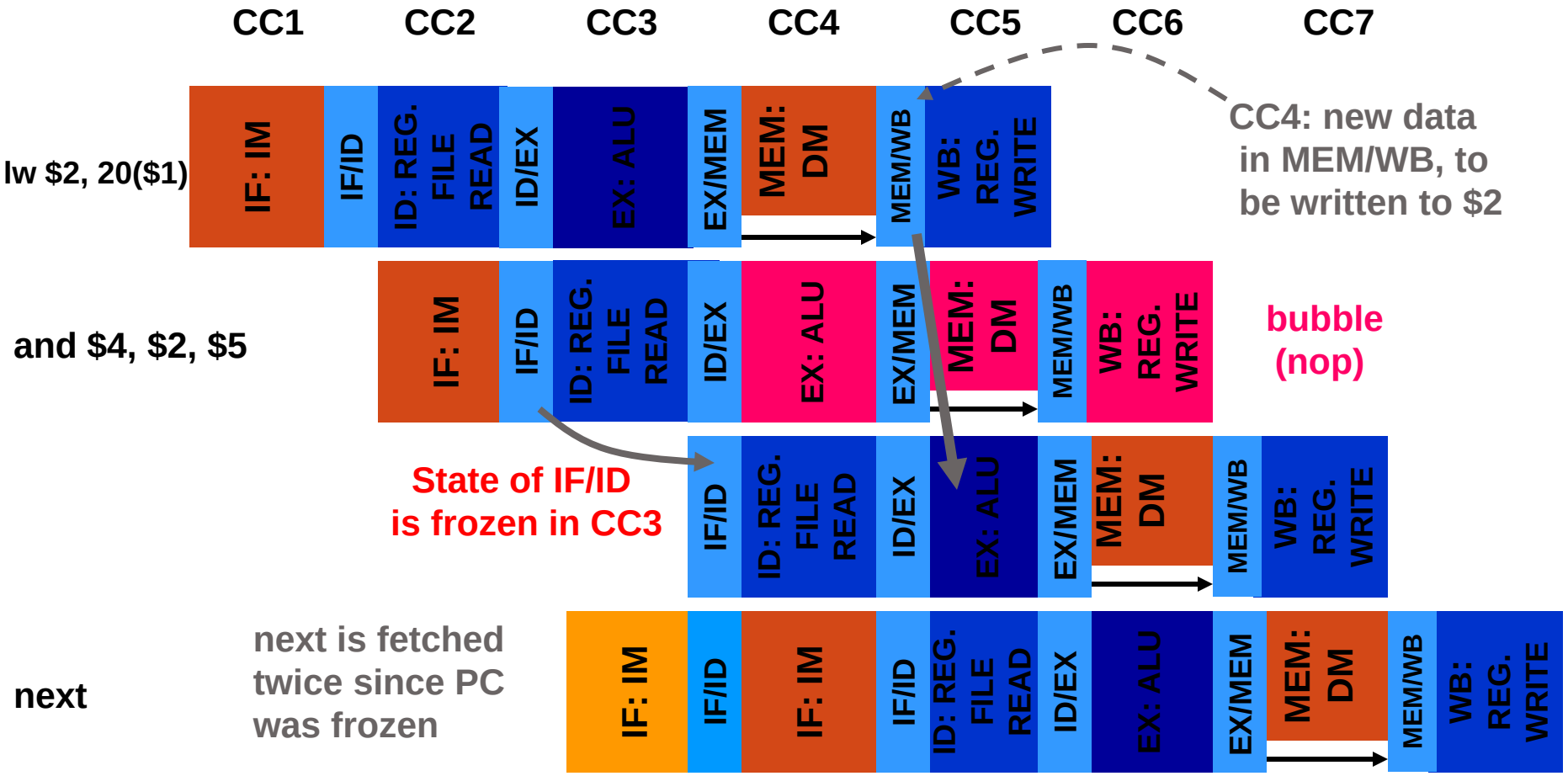
- Hazards are resolved by Hazard detection and forwarding units.
- Compiler's understanding of how these units work can improve performance.

Detecting Hazard Requiring Stall

- Consider instruction in IF/ID being decoded:
- *If*
  - Previous instruction (lw) activated MemRead, and
  - Instruction being decoded has a source register (Rs or Rt) same as the destination register (Rt for lw) of the previous instruction
- *Then*, stall the pipeline:
  - Force all control outputs to 0
  - Prevent PC from changing
  - Prevent IF/ID from changing

Stall and Forwarding

- Execution with stall and forwarding:



Avoiding Stall by Code Reorder

C code:

```
A = B + E;
C = B + F;
```

MIPS code:

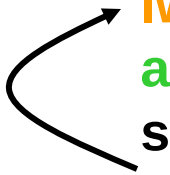
```
1   lw   $t1,  0($t0)
2   lw   $t2,  4($t0)
3   add  $t3,  $t1, $t2   # $t2 not yet written (cycle 5 of prev)
4   sw   $t3,  12($t0)
5   lw   $t4,  8($t0)
6   add  $t5,  $t1, $t4   # $t4 not yet written (cycle 5 of prev)
7   sw   $t5,  16($t0)
```

Compiler Reordered Code

C code:

```
A = B + E;
C = B + F;
```

MIPS code:



```
1   lw    $t1,   0($t0)
2   lw    $t2,   4($t0)
5   lw    $t4,   8($t0)
3   add   $t3,   $t1, $t2     no hazard
4   sw    $t3,   12($t0)
6   add   $t5,   $t1, $t4     no hazard
7   sw    $t5,   16($t0)
```

Control or Branch Hazard

- Instruction to be fetched is not known!
- Example: Instruction being executed is branch-type, which will determine the next instruction:

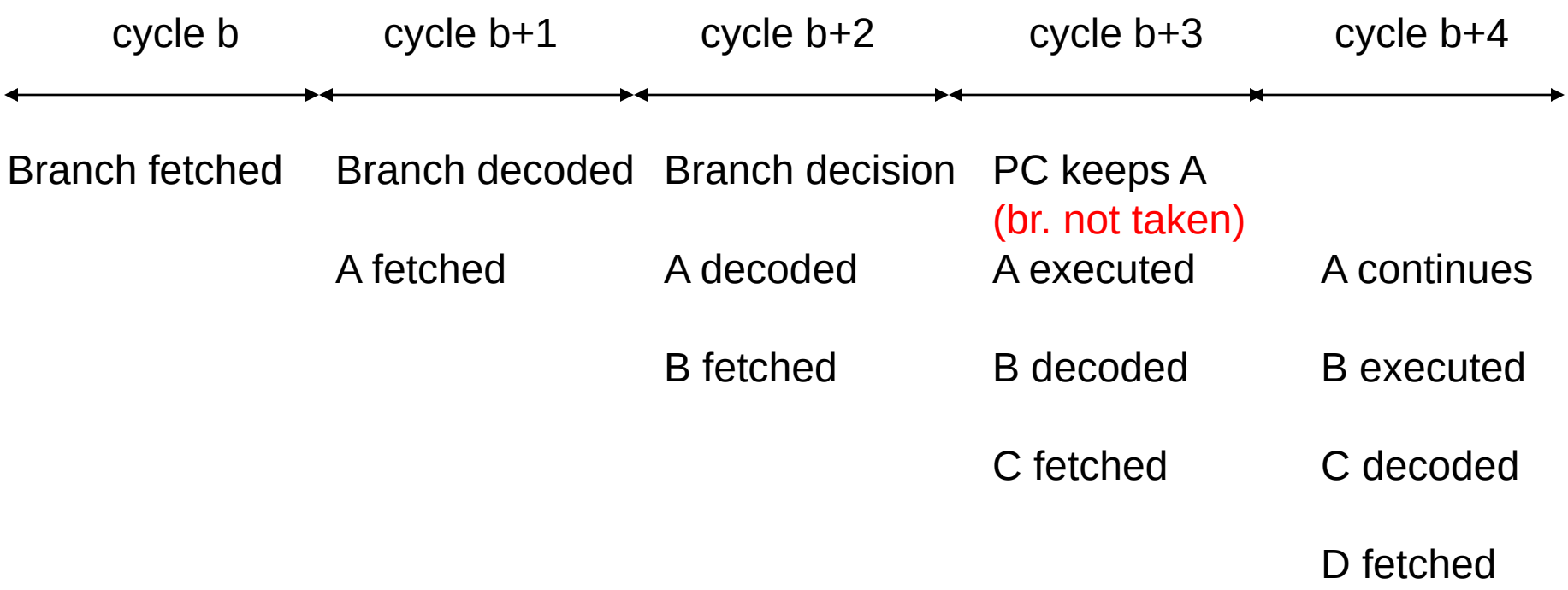
```
            add   $4, $5, $6
            beq   $1, $2, 40
            next instruction ??
            . . .
40          and   $7, $8, $9
```

Branch Hazard

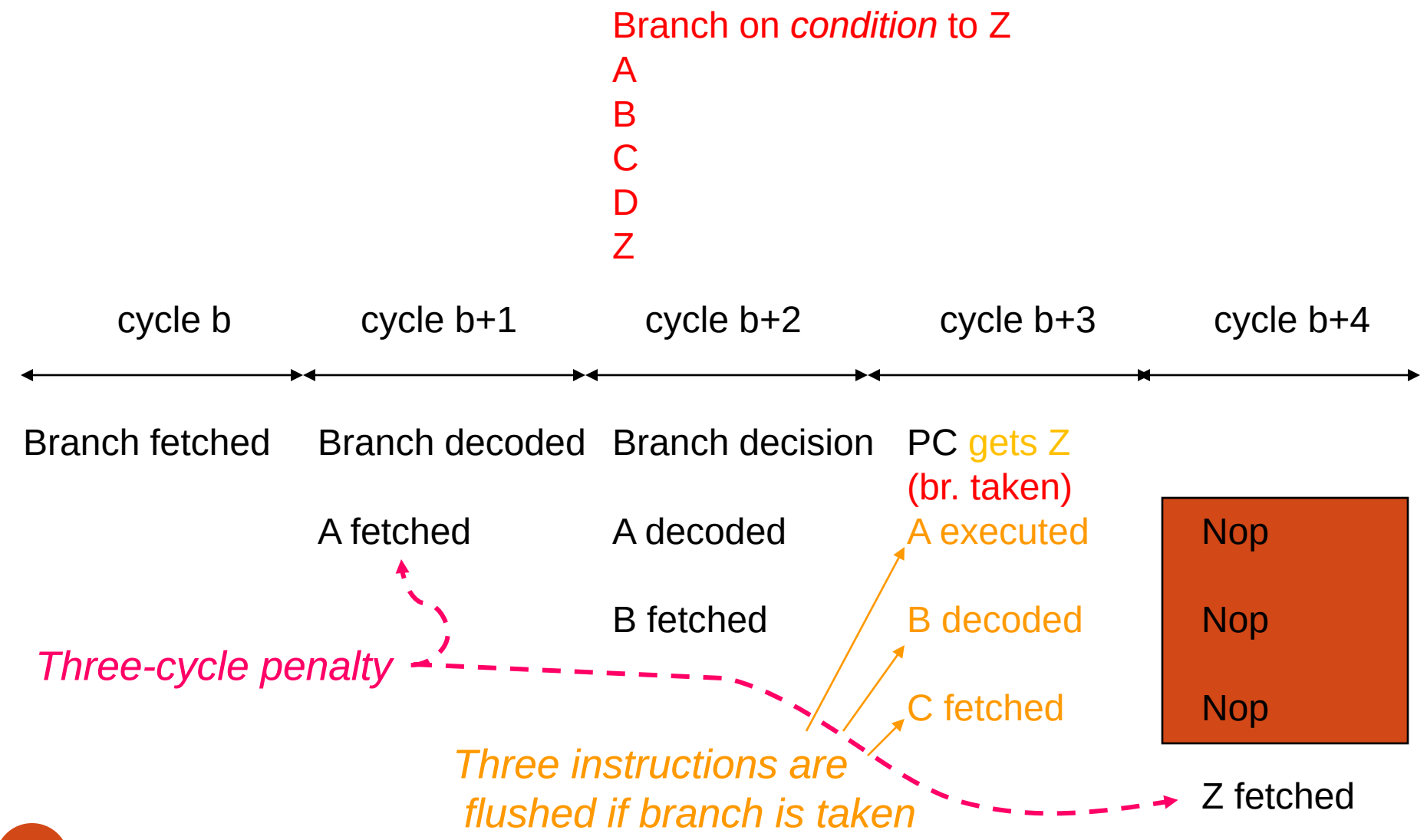
- Consider heuristic – branch not taken.
- Continue fetching instructions in sequence following the branch instructions.
- If branch is taken (indicated by *zero* output of ALU):
  - Control generates *branch* signal.
  - *branch* activates *PCSource* signal in the MEM cycle to load PC with new branch address.
  - *Three instructions in the pipeline must be flushed if branch is taken – can this penalty be reduced?*

Branch Not Taken

Branch on *condition* to Z
A
B
C
D
Z



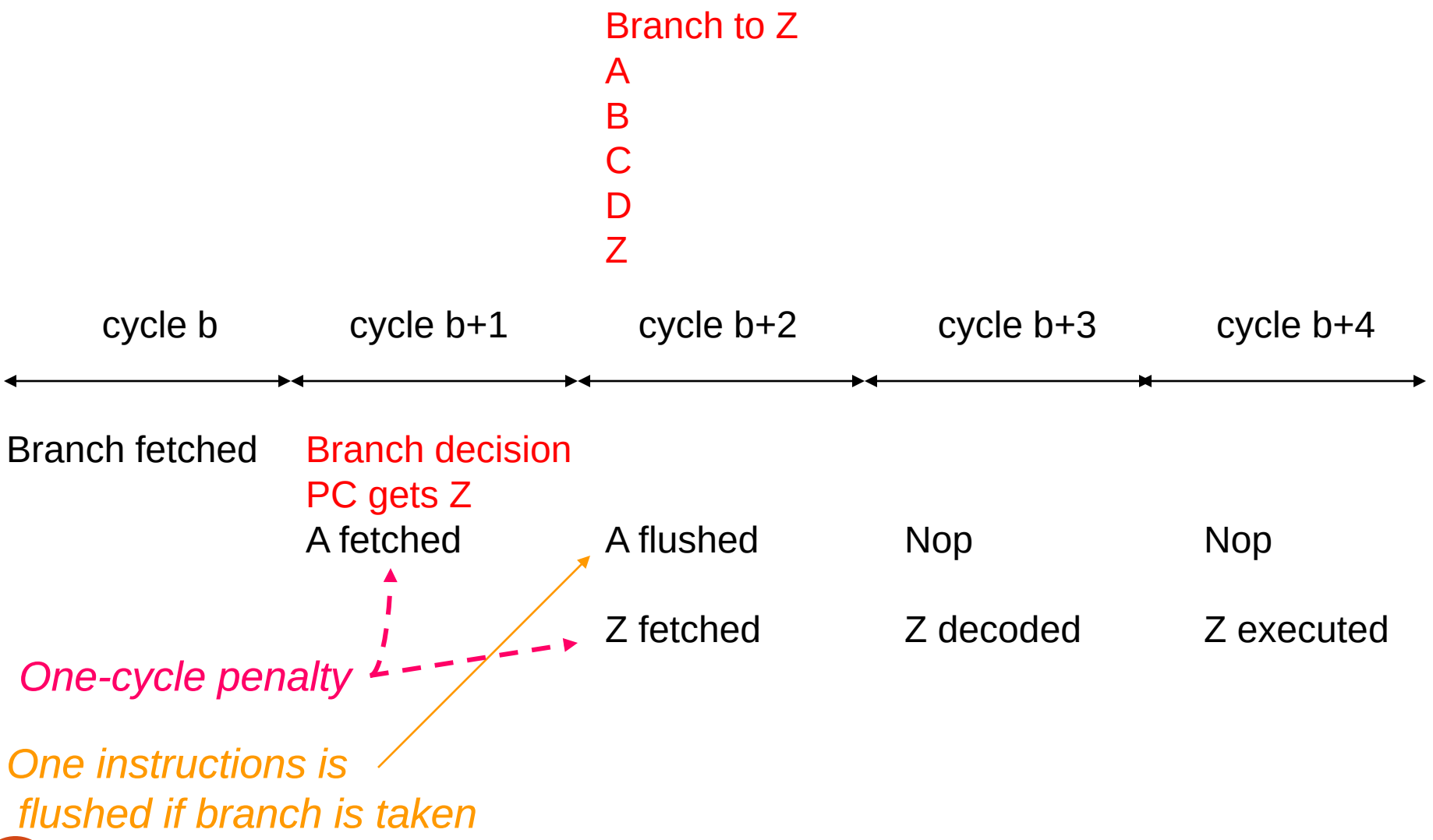
Branch Taken



Pipeline Flush

- If branch is taken (as indicated by *zero*), then control does the following in :pipeline flushing
  - Change all control signals to 0, similar to the case of stall for data hazard, i.e., insert bubble in the pipeline.
  - Generate a signal *IF.Flush* that changes the instruction in the pipeline register IF/ID to 0 (nop).
- Penalty of branch hazard is reduced by
  - Adding branch detection and address generation hardware in the decode cycle – **one bubble needed** – *a next address generation logic in the decode stage writes PC+4, branch address, or jump address into PC.*

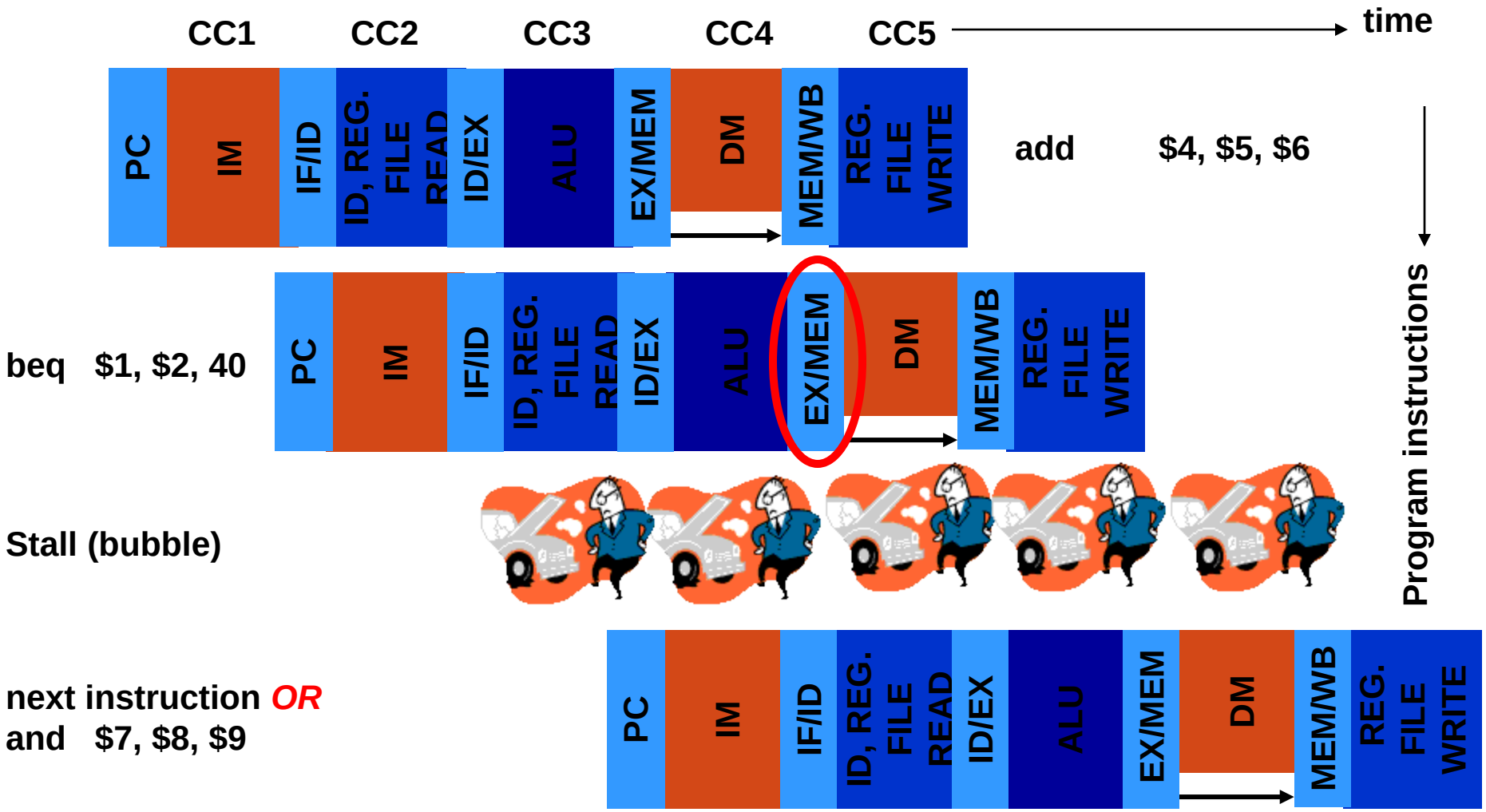
Branch Taken: Reduce Flush Penalty



Ways to Handle Branch

- Stall or bubble (extra hardware in ID)
- Delayed branch
- Branch prediction:
  - Heuristics
    - Next instruction (fixed)
    - Prediction based on statistics (dynamic)
    - Hardware decision (dynamic)
  - Prediction error: pipeline flush

Stall on Branch



Why Only One Stall?

- Extra hardware in ID phase:
  - Additional ALU to compute branch address
  - Comparator to generate zero signal
  - Hazard detection unit writes the branch address in PC

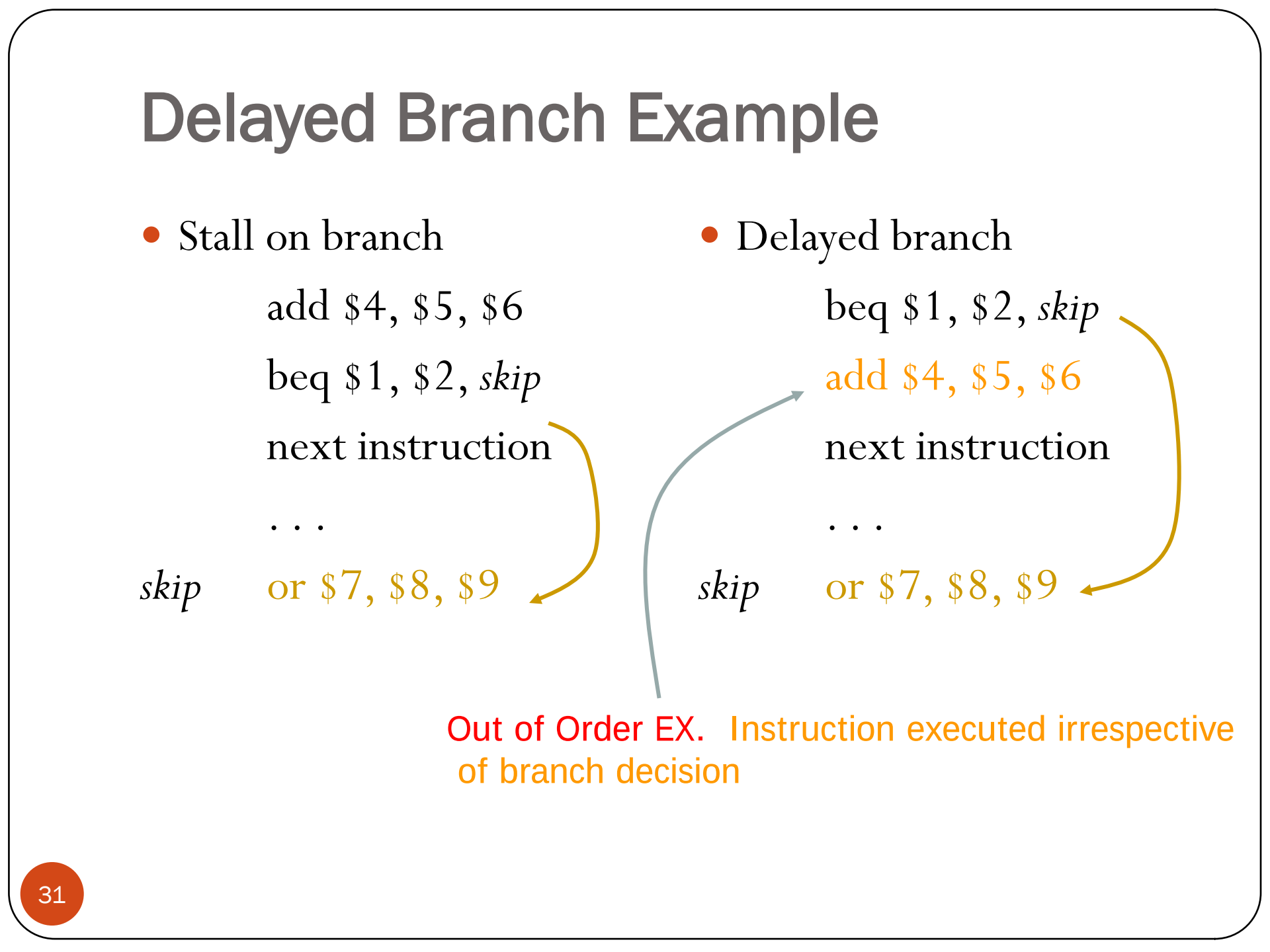
Delayed Branch Example

- Stall on branch

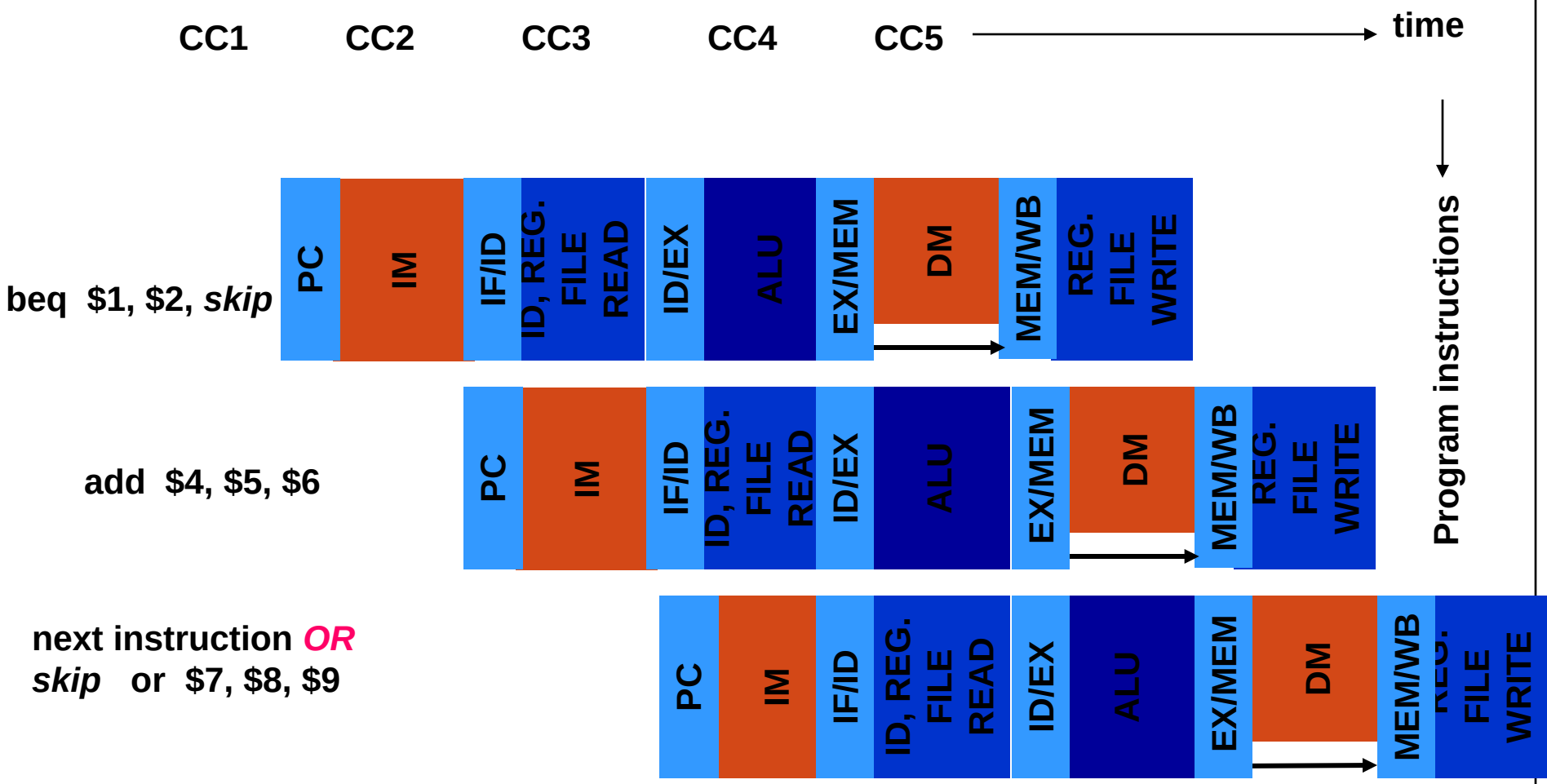
```
        add $4, $5, $6
        beq $1, $2, skip
        next instruction
        . . .
skip    or $7, $8, $9
```

- Delayed branch

```
        beq $1, $2, skip
        add $4, $5, $6
        next instruction
        . . .
skip    or $7, $8, $9
```

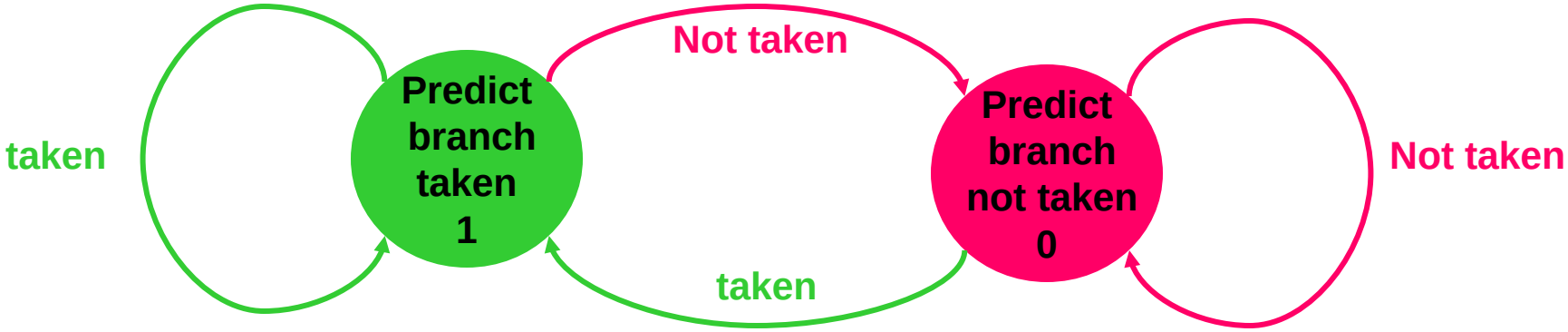


Delayed Branch

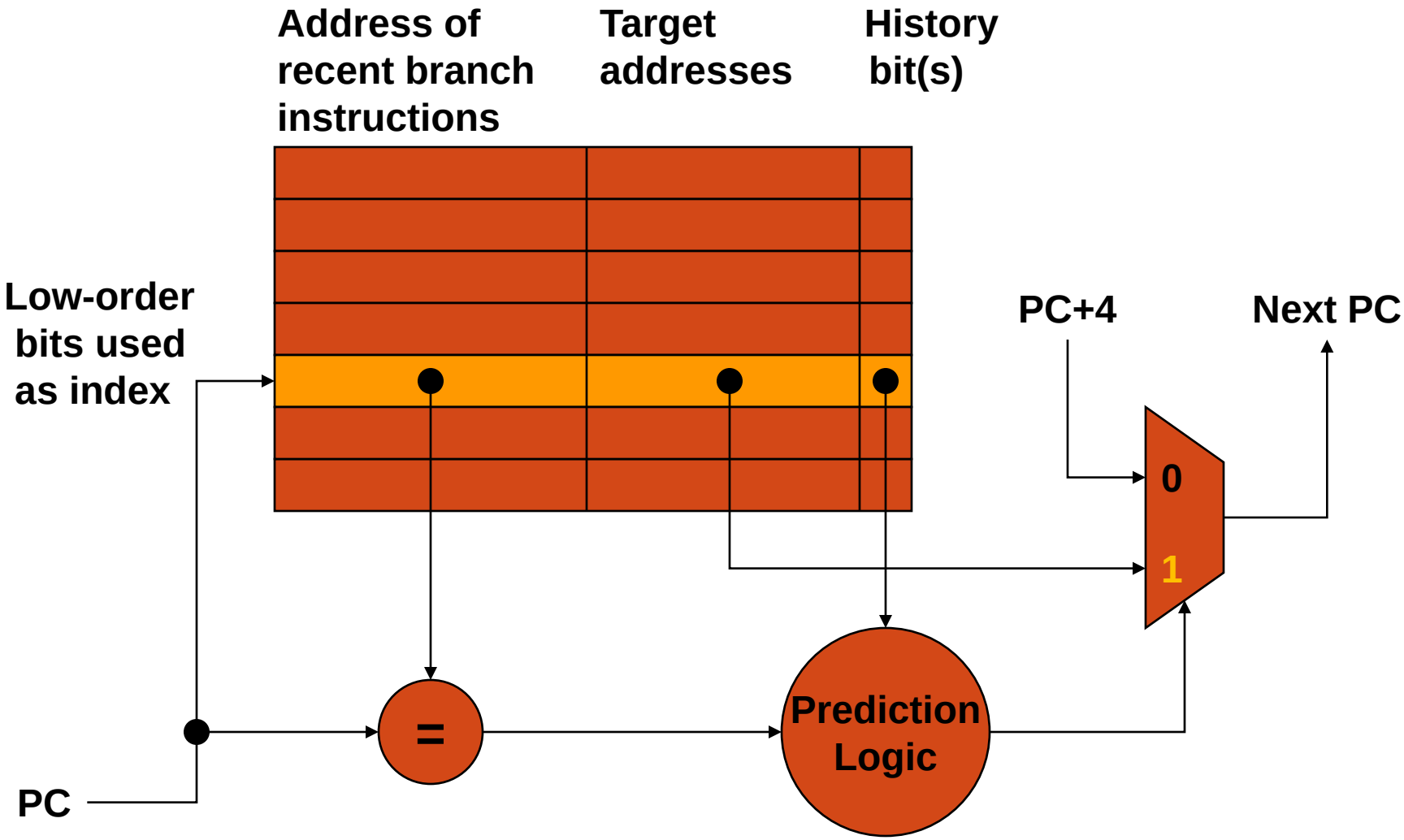


Branch Prediction

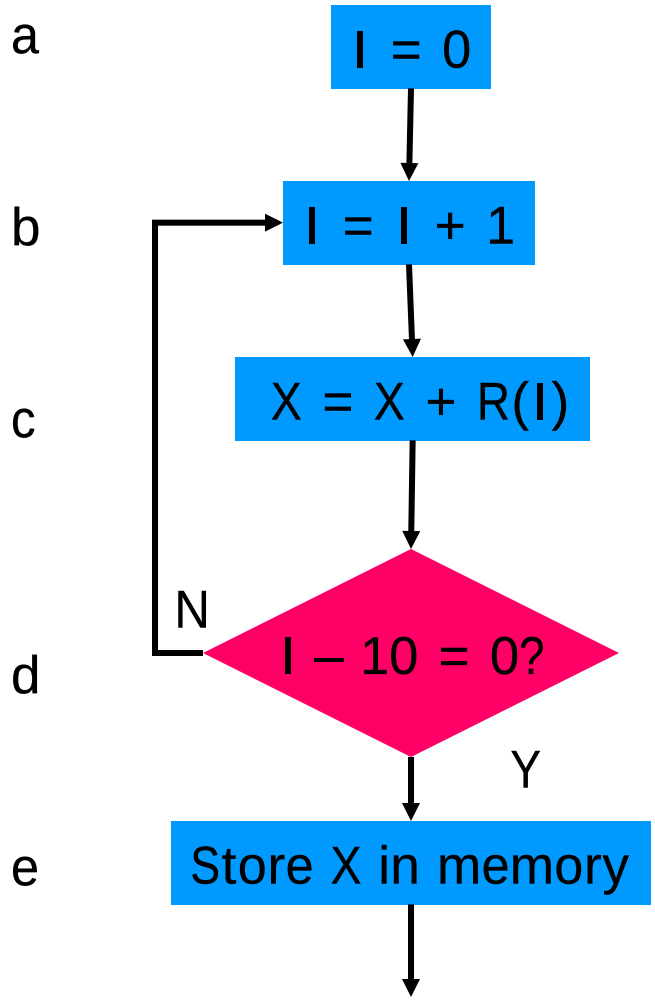
- Useful for program loops.
- A one-bit prediction scheme: a one-bit buffer carries a "history bit" that tells what happened on the last branch instruction
  - History bit = 1, branch was taken
  - History bit = 0, branch was not taken



Branch Prediction



Branch Prediction for a Loop



Execution of Instruction d

Execu-tion seq.	Old hist. bit	Next instr.			New hist. bit	Predi ction
		Pred.	Iter	Act.		
1	0	e	1	b	1	Bad
2	1	b	2	b	1	Good
3	1	b	3	b	1	Good
4	1	b	4	b	1	Good
5	1	b	5	b	1	Good
6	1	b	6	b	1	Good
7	1	b	7	b	1	Good
8	1	b	8	b	1	Good
9	1	b	9	b	1	Good
10	1	b	10	e	0	Bad

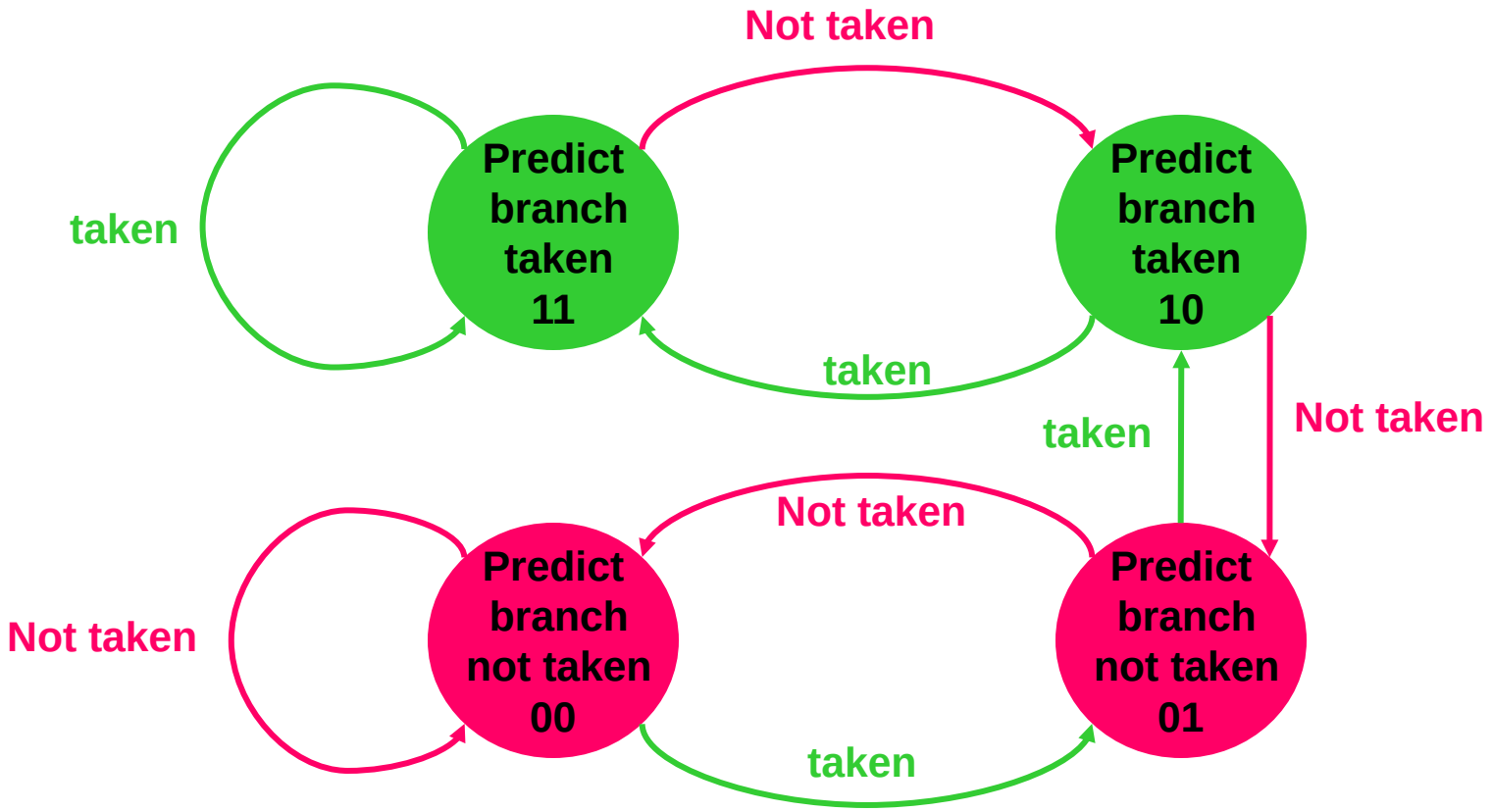
h.bit = 0 *branch not taken*, h.bit = 1 *branch taken*.

Branch Prediction Accuracy

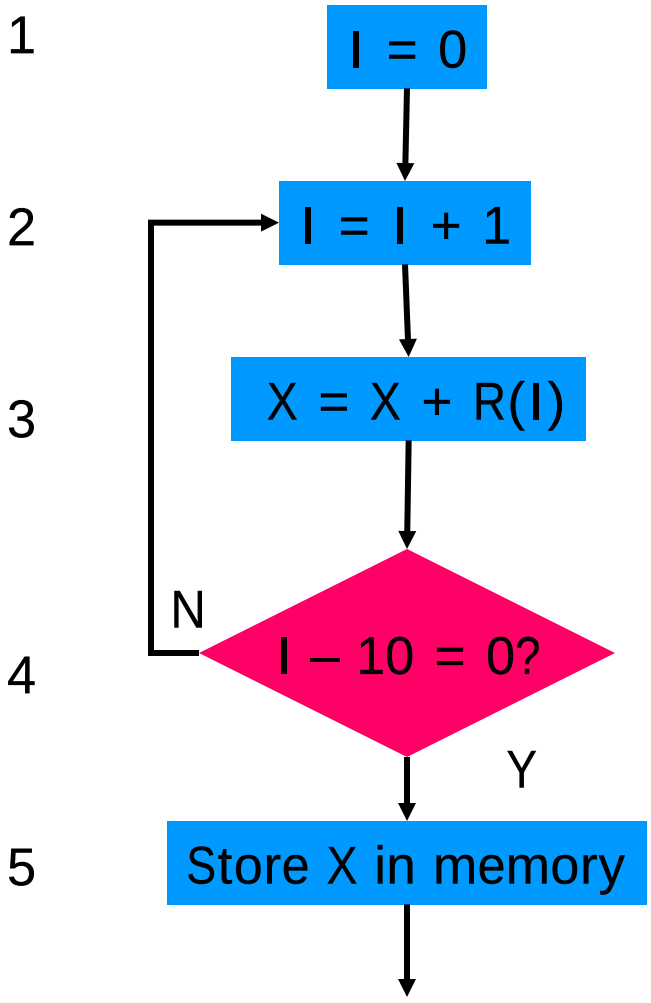
- One-bit predictor:
  - 2 errors out of 10 predictions
  - Prediction accuracy = 80%
- To improve prediction accuracy, use two-bit predictor:
  - A prediction must be wrong twice before it is changed

Two-Bit Prediction Buffer

- Implemented as a two-bit counter.
- Can improve correct prediction statistics.



Branch Prediction for a Loop



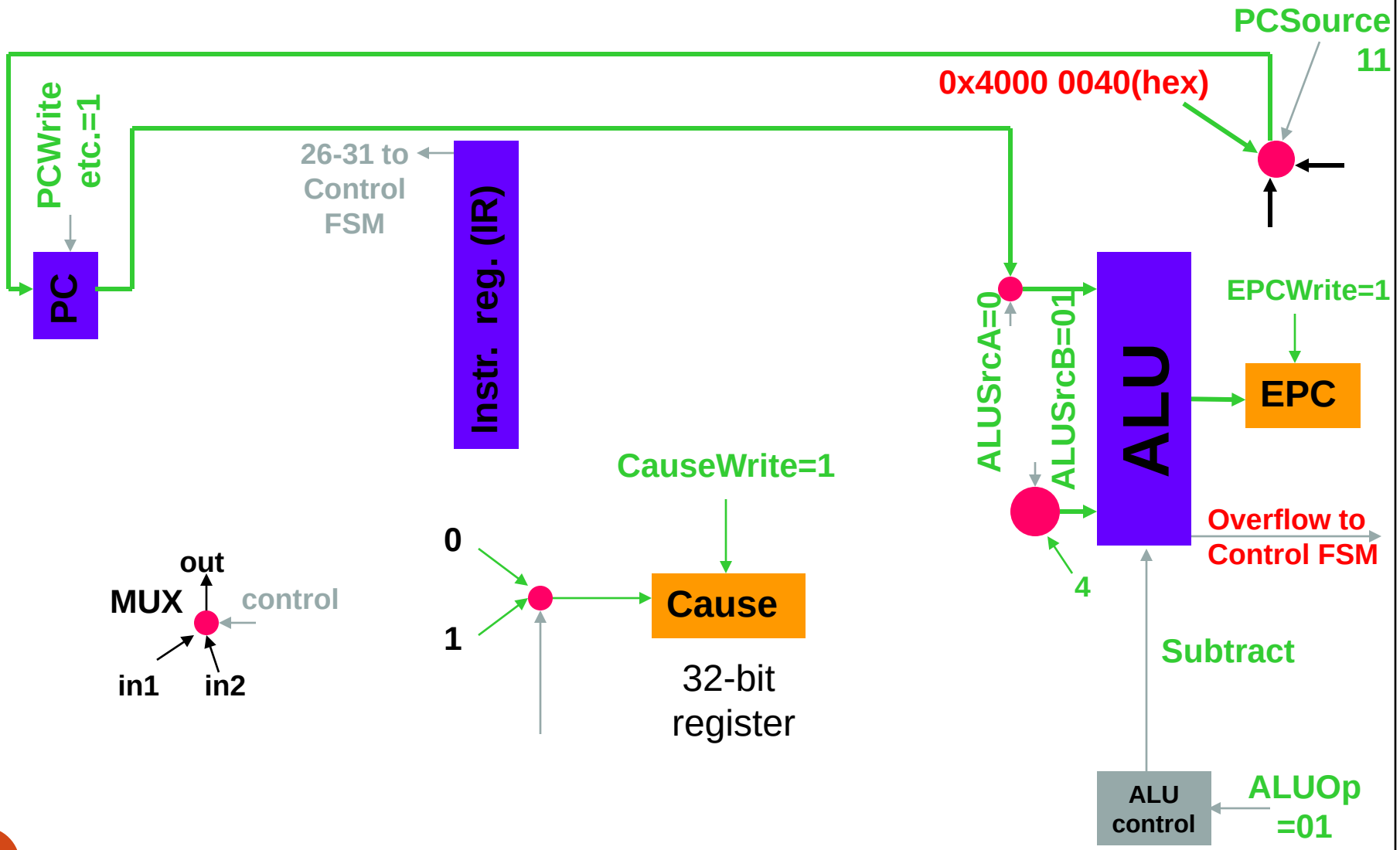
Execution of Instruction 4

Execu-tion seq.	Old Pred. Buf	Next instr.			New pred. Buf	Predi ction
		Pred.	Iter	Act.		
1	10	2	1	2	11	Good
2	11	2	2	2	11	Good
3	11	2	3	2	11	Good
4	11	2	4	2	11	Good
5	11	2	5	2	11	Good
6	11	2	6	2	11	Good
7	11	2	7	2	11	Good
8	11	2	8	2	11	Good
9	11	2	9	2	11	Good
10	11	2	10	5	10	Bad

Exceptions or Interrupts

- Conditions under which the processor may produce incorrect result or may "hang".
  - Illegal or undefined opcode.
  - Arithmetic overflow, divide by zero, etc.
  - Out of bounds memory address.
- EPC: 32-bit register holds the affected instruction address.
- Cause: 32-bit register holds an encoded exception type. For example,
  - 0 for undefined instruction
  - 1 for arithmetic overflow

Implementing Exceptions



Exceptions

- A typical exception occurs when ALU produces an *overflow* signal.
- Control asserts following actions on exception:
  - Change the PC address to 4000 0040hex. This is the location of the exception routine. This is done by adding an additional input to the PC input multiplexer.
  - Overflow is detected in the EX cycle. Similar to data hazard and pipeline flush,
    - Set IF/ID to 0 (nop).
    - Generate ID.Flush and EX.Flush signals to set all control signals to 0 in ID/EX and EX/MEM registers. This also prevents the ALU result (presumed contaminated) from being written in the WB cycle.

Summary: Hazards

- Structural hazards
  - Cause: resource conflict
  - Remedies: (i) hardware resources, (ii) stall (bubble)
- Data hazards
  - Cause: data unavailablity
  - Remedies: (i) forwarding, (ii) stall (bubble), (iii) code reordering (compiler based)
- Control hazards
  - Cause: out-of-sequence execution (branch or jump)
  - Remedies: (i) stall (bubble)/ pipeline flush, (ii) delayed branch/pipeline flush, (iii) branch prediction/pipeline flush